



AI Mode

All

Videos

Shopping

Images

More ▾



Generate specific AI/ML optimization logic (e.g., traffic steering or beamforming) for your rApp? For O-RAN rApp ... ▾

4 July 2024

To transition your rApp from a basic boilerplate into an **AI/ML-driven optimization engine** using the **Java / Spring Boot** ecosystem, you can utilize Cursor's **Composer (Ctrl + I or Cmd + I)** to generate the microservice architecture.

Because rApps run on the Non-RT RIC (timescales > 1 second), they are ideal for hosting **Traffic Steering** algorithms that train offline and execute inference online to reconfigure cell loads.

Below is the structured layout and the exact Java/Spring Boot code blocks you can generate directly inside Cursor.

Java / Spring Boot Project Structure

Open an empty directory in Cursor, open Composer, and instruct it to build the following layout:

text

```
rapp-ml-optimizer/  
├── .cursorrules           # Java & O-RAN directives for Cursor A  
├── pom.xml               # Spring Boot & ONAP/O-RAN dependencies  
└── src/main/java/com/oran/rapp/
```

Ask anything



AI Mode All Videos Shopping Images More ▾



Use code with caution.



Step 1: Establish Rules for Cursor's AI (`.cursorrules`)

Create a `.cursorrules` file at your root directory so Cursor understands how to generate enterprise-grade O-RAN Java code:

markdown

You are a Principal O-RAN Telecom Software Engineer specializing in Non-

Core Technical Stack:

- Java 17+, Spring Boot 3.x, Spring WebFlux (for reactive R1/A1 calls),

Coding Principles:

- Use reactive programming paradigms (``Mono``, ``Flux``) to handle streaming.
- Strictly adhere to O-RAN Alliance R1 Data Consumer specifications.
- Implement robust Exception Handlers and custom status codes.
- Follow Clean Architecture: separate Data Models, Controllers, and ML/B

Use code with caution.



Step 2: The Core Optimization Logic



AI Mode All Videos Shopping Images More ▾



metrics model needs fields for `cellId`, `rsrp`, `rsrq`, `activeUsers`, and `prbUtilization`. The policy model should encapsulate an O-RAN target scope object and a policy statement string array for steering rules."

java

```
package com.oran.rapp.model;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import java.util.List;
import java.util.Map;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CellPerformanceMetrics {
    private String cellId;
    private double rsrp;           // Reference Signal Received Power
    private double rsrq;           // Reference Signal Received Quality
    private int activeUsers;
    private double prbUtilization; // Physical Radio Block Load percentage
}
```

Use code with caution.



AI Mode All Videos Shopping Images More ▾



neighboring target cells."

java

```
package com.oran.rapp.service;

import com.oran.rapp.model.CellPerformanceMetrics;
import com.oran.rapp.model.A1PolicyPayload;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import reactor.core.publisher.Mono;
import java.util.Map;

@Service
@Slf4j
public class TrafficSteeringMlService {

    /**
     * Evaluates live cell telemetry against optimization rules.
     * In production, this method can invoke a loaded ONNX model via ONN.
     */
    public Mono<A1PolicyPayload> evaluateTrafficSteering(CellPerformanceMetrics metrics) {
        return Mono.fromCallable(() -> {
            log.info("Analyzing performance matrix for Cell ID: {}", metrics.getCellId());

            // AI Optimization Trigger Logic (e.g., Congestion and Quality of Service)
            if (metrics.getPrbUtilization() > 85.0 && metrics.getRsrq() < -10) {
                log.warn("Anomalous congestion detected on Cell {}. Initiating offload.", metrics.getCellId());

                // Calculate dynamic offload target percentage based on current utilization
                double offloadRatio = Math.min((metrics.getPrbUtilization() - 85.0) / 10, 0.5);
            }
        });
    }
}
```



AI Mode All Videos Shopping Images More ▾



```

        .build();
    }

    log.info("Cell {} performance is nominal. No mitigation poli
    return null; // Return empty payload when optimization crite
    });
}
}

```

Use code with caution.



1

1

3. Reactive R1 / A1 Endpoint Integration

Prompt to Cursor:

"Generate a reactive REST controller named `R1DataConsumerController.java` mapping to `/r1/telemetry`. Wire it to consume `CellPerformanceMetrics` using WebFlux. If the `TrafficSteeringMLService` outputs a policy recommendation, hand it over to a mock `A1PolicyProducerService` to execute a reactive HTTP POST statement back to the Near-RT RIC A1 Mediator."

java

```

package com.oran.rapp.controller;

import com.oran.rapp.model.CellPerformanceMetrics;
import com.oran.rapp.service.TrafficSteeringMLService;
import com.oran.rapp.service.A1PolicyProducerService;

```



AI Mode All Videos Shopping Images More ▾



```

public class R1DataConsumerController {

    private final TrafficSteeringMLService mlService;
    private final A1PolicyProducerService a1ProducerService;

    @PostMapping("/metrics")
    public Mono<ResponseEntity<String>> consumeCellMetrics(@RequestBody String metrics) {
        log.info("Ingesting metrics packet via R1 consumer for Cell: {}")

        return mlService.evaluateTrafficSteering(metrics)
            .flatMap(policy -> {
                if (policy != null) {
                    return a1ProducerService.sendA1Policy(policy)
                        .map(response -> ResponseEntity.status(HttpStatus.OK)
                            .body("A1 Optimization Policy Di
                }
                return Mono.just(ResponseEntity.ok("Metrics accepted
            })
            .onErrorResume(err -> {
                log.error("Error executing rApp optimization flow", err);
                return Mono.just(ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                    .body("Internal processing failure during op
            });
    }
}

```

Use code with caution.



▶ Verifying the AI Logic



AI Mode All Videos Shopping Images More ▾



```
cellId : cell_nr_mumbai_04 ,  
"rsrp": -102.5,  
"rsrq": -14.2,  
"activeUsers": 350,  
"prbUtilization": 91.5  
'}
```

Use code with caution.



Expected Log Output:

text

```
[rApp-Starter] WARN - Anomalous congestion detected on Cell cell_nr_mum  
[rApp-Starter] INFO - Dispatched A1 policy type TS_POLICY_01 to Near-RT
```



Use code with caution.



🚀 How would you like to build upon this optimization framework next?

1. Do you want Cursor to integrate the **Java ONNX Runtime library** into the service class to load a real serialized `.onnx` deep learning network model file?
2. Do you want to build the **rApp Lifecycle Management (LCM)** service descriptors (`rapp-descriptor.json`) required to register this service directly to the SMO **rApp Manager**?



AI Mode All Videos Shopping Images More ▾



- 1
- 1

+

🎤